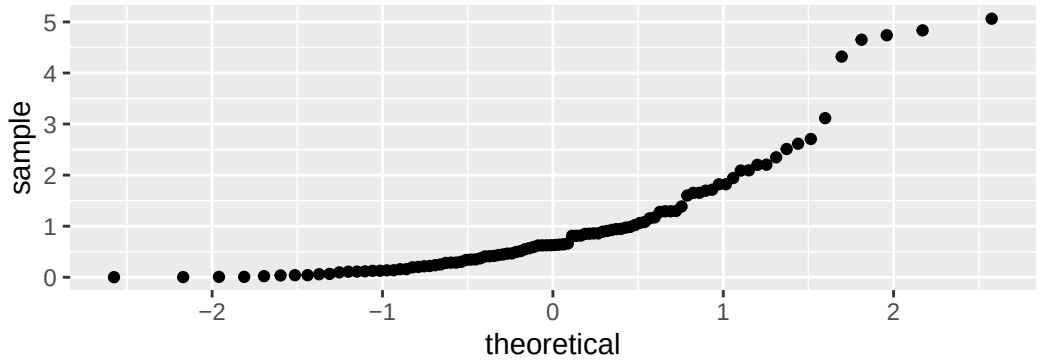
# Stat 343 Intro to Quarto

Thomas Scofield

## An investigation of normal-quantile plots

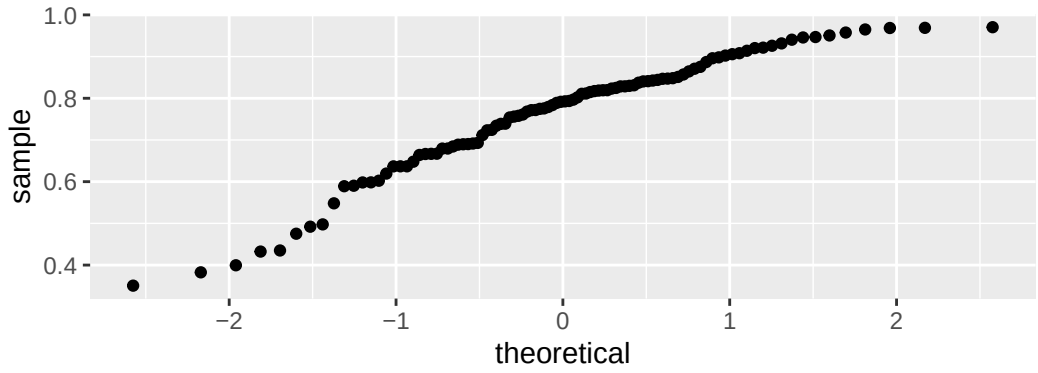
Exercise 3.44 in the FastR2 text seems to suggest the shape of a normal-quantile plot may correspond to certain shapes in the underlying distribution. We investigate, first, what sort of normal-quantile plot arises from data that is from a right-skewed population. I select an exponential distribution, draw a random sample, and then look at the corresponding normal quantile plot.

```
mydata = rexp(100, 1)
gf_qq(~mydata)
```




For left-skewed data, like that coming from a beta distribution with shqpe1 = 6 and shape2 = 2, the normal quantile plot is

```
mydatabeta = rbeta(100, shape1=6, shape2=2)
gf_qq(~mydatabeta)
```




## Simulating probabilities

### A roll-die-then-draw-token simulation

First, I want to simulate the die-roll-token-draw probabilities. To do it once, I might execute this code chunk.

```
bagA = c(0,0,0,0,1,1)   # 4 Red, 2 Green tokens
bagB = c(0,1,1,1,1,1)   # 1 Red, 5 Green
dieRoll = sample(c(1:3), size=1)
if (dieRoll == 1) {token=sample(bagA, size=1)} else {
  token=sample(bagB, size=1) }
token
```

```
[1] 1
```

We do it many times in order to learn, via simulation, the probability of drawing a green token:

```
result = replicate(5000,
 {dieRoll = sample(c(1:3), size=1);
 if (dieRoll == 1) {token=sample(c(0,0,0,0,1,1), size=1)} else {
  token=sample(c(0,1,1,1,1,1), size=1) };
 token}
)
sum(result)/5000
```

```
[1] 0.6656
```

Following these 5000 steps, I got 3328 greens.

### Simulating a straight flush

There must be many ways to do this. My approach will be to think of the numbers 1–52 in groups of 13. In alphabetical order, perhaps the *clubs* are 1–13, *diamonds* 14-26, *hearts* 27-39, and *spades* 40-52. In that vein, a hand dealt to me might be

```
hand = sample(1:52, size=5)
sort(hand)
```

```
[1] 17 19 37 47 52
```

which translates as the

- 4 of diamonds
- 6 of diamonds
- J of hearts
- 8 of spades
- K of spades

(Here, I am treating the aces as 1, 14, 27, and 40.)

The main algorithm will be this:

```
hand = sort(sample(c(1:52), size=5))
suits = round(hand/13)
flush = (suits[5] == suits[1])
straight = (hand[5] == hand[1] + 4)
```

If both `hand` and `flush` are TRUE, we have a straight flush. The exception would be the *royal flush*, which treats an ace as the last card in a run 10-J-Q-K-A, such as in 1-10-11-12-13, or 40-49-50-51-52. We add another line to check if we have one of these:

```
(hand[5] == hand[2] + 3) & (hand[5] == hand[1] + 12)
```

```
[1] FALSE
```

We might define a function to which `hand` is sent, and checks for both the flush condition and the straight.

```
straightFlush = function(hand) {
  hand = sort(hand)
  suits = round(hand/13)
  flush = (suits[5] == suits[1])
  straight = (hand[5] == hand[1] + 4) |
    ((hand[5] == hand[2] + 3) &
       (hand[5] == hand[1] + 12))
  return (flush & straight)
}

result = replicate(10000000, {
  hand = sample(c(1:52), size=5)
  straightFlush(hand)
})
sum(result)/10000000
```

```
[1] 1.48e-05
```

The simulated probability of getting a straight-flush is $1.48 \times 10^{-5}$. Wikipedia says the chance of a straight-flush is 0.00154%. Of course, each new run of the code chunk above will produce a different result. By the **law of large numbers**, more repetitions should bring better approximations of the *true* probability, but to do more than 10 million requires even greater patience.

### Simulating a hypergeometric probability

The table on p. 99 gives rise to a data frame:

```
twins <- rbind(
  do(2) * c("dizygotic", "convicted"),
  do(15) * c("dizygotic", "notConvicted"),
  do(10) * c("monozygotic", "convicted"),
  do(3) * c("monozygotic", "notConvicted")
)
colnames(twins) = c("type", "record")
head(twins)
```

```
       type       record
1 dizygotic    convicted
2 dizygotic    convicted
```

```
3 dizygotic notConvicted
4 dizygotic notConvicted
5 dizygotic notConvicted
6 dizygotic notConvicted
```

We use this raw data to reconstruct the table:

```
tally(type ~ record, data=twins)
```

```
             record
type          convicted notConvicted
  dizygotic           2           15
  monozygotic        10            3
```

The number convicted among monozygotic twins is 10, and accessed from the array/table as

```
tally(type ~ record, data=twins)[2,1]
```

```
[1] 10
```

It seems somewhat on the high side, particularly when you view there are only 13 monozygotic twins in the study, and the number of convicted dizygotic twins is a small proportion of the 17 of those in the study.

But what hypotheses are we testing? It's

$$\mathbf{H}_0 : \text{'Type of twin' and 'whether convicted' are independent.}$$

It seems the alternative one would set out to prove is one-sided, reflecting a higher conviction rate among monozygotic twins. In that regard, our $P$-value should reflect results as extreme or more so than 10 in the bottom left cell. That is the value produced by the command

```
1-phyper(9, 13, 17, 12)
```

```
[1] 0.0004651809
```

We might simulate this probability using **permutation testing**. We maintain the marginal totals, but shuffle one variable. To do so, changes the counts in the table itself. Here is one run.

```
tally(type ~ shuffle(record), data=twins)
```

```
             shuffle(record)
type          convicted notConvicted
  dizygotic           7           10
  monozygotic         5            8
```

When we repeat this many times, we will grab just the value in the 2nd row, 1st column.

```
simCornerValue <- replicate(50000,
          tally(type ~ shuffle(record), data=twins)[2,1]
          )
head(simCornerValue)
```

```
[1] 7 6 7 4 2 4
```

We find the proportion of values higher than 10:

```
prop(~(simCornerValue >= 10))
```

```
prop_TRUE
  0.00056
```

This is reasonably close to the value 0.0004652 found earlier using `phyper()`.